

Laboratorio 10: Aprendizaje Semi-Supervisado

Bryan Martínez, Adriana Palacios y Nery Molina

2026-05-17

Introducción

El presente informe desarrolla el décimo laboratorio del curso de Minería de Datos, en donde se tiene como enfoque experimentar y profundizar en el funcionamiento de los algoritmos de aprendizaje semi-supervisado. En este caso, se trabajará con el conjunto de datos *Adult Income Prediction Dataset*, obtenido desde Kaggle a través del siguiente enlace: <https://www.kaggle.com/datasets/mosapabdelghany/adult-income-prediction-dataset>.

El objetivo principal es contrastar el desempeño de un modelo supervisado tradicional contra dos modelos semi-supervisados. Esta comparación es posible porque el conjunto de datos cuenta con etiquetas conocidas para la variable objetivo, lo que permite simular un escenario semi-supervisado ocultando una proporción de dichas etiquetas durante el entrenamiento. De esta manera, se podrá analizar cómo se comportan los modelos cuando únicamente disponen de una fracción reducida de datos etiquetados y el resto de observaciones se trata como información no etiquetada.

Metodología

Para el desarrollo del laboratorio se va a estar utilizando el dataset *Adult Income Prediction*, obtenido desde Kaggle. Este dataset proporciona información demográfica, educativa y laboral de personas adultas, con el objetivo de predecir si su ingreso anual es menor o igual a \$50,000 o mayor a \$50,000. Esto implica que se estará trabajando con un problema de clasificación en donde se analizará si la persona gana más o menos de la cantidad mencionada anteriormente. Esto se evaluará a través de la variable objetivo `income`.

Para realizar el análisis, primero se hará un análisis exploratorio sobre el conjunto de datos, validando su dimensionalidad, distribución de las clases, variables con las que se está trabajando, presencia de valores faltantes, correlaciones, entre otros aspectos. Posteriormente, se aplicarán las transformaciones de datos necesarias para poder trabajar con el modelo supervisado y los otros dos modelos semi-supervisados a utilizar.

Asimismo, es importante mencionar que el dataset ya tiene las respuestas de qué clasificación le corresponde a cada una de las observaciones. Dicho esto, para simular un escenario de aprendizaje semi-supervisado, el conjunto de datos primero va a ser dividido en entrenamiento y prueba. A partir del conjunto de entrenamiento se conservará únicamente el 10% de las etiquetas, mientras que el resto será tratado como aquellas que no están disponibles en el dataset. Finalmente, se contrastará el desempeño entre el modelo supervisado y los dos modelos semi-supervisados a implementar.

Los algoritmos con los que se va a estar trabajando para el aprendizaje semi-supervisado serán *Label Propagation* y k-means con restricciones. Dadas estas elecciones, al ser algoritmos que dependen de medidas de distancia para evaluar la similitud entre las observaciones, fue necesario aplicar un proceso de limpieza y transformación previo al entrenamiento.

En primer lugar, todas las observaciones que en alguna variable tenían un signo de interrogación serán tratadas como valores desconocidos y se les asignará una categoría adicional. Asimismo, dada la naturaleza de algunas variables, la variable `education` será excluida, ya que su información ya se ve representada de forma ordinal a través de la variable `education.num`.

De igual forma, la variable `fnlwgt` no será utilizada como variable predictora, ya que representa información estadística asociada al censo realizado y no corresponde directamente a una característica de la persona que aporte valor al análisis de clasificación. Esta decisión busca evitar que variables poco interpretables afecten las medidas de distancia utilizadas por los algoritmos a implementar.

Las variables `capital.gain` y `capital.loss` serán transformadas mediante `log1p`, debido a que presentan una alta concentración de valores en cero y algunos valores extremos. Posteriormente, las variables numéricas serán estandarizadas y las variables categóricas serán trabajadas mediante *one-hot encoding*.

Análisis exploratorio de datos

El conjunto de datos seleccionado cumple con las condiciones necesarias para el desarrollo del laboratorio, ya que corresponde a un dataset real, disponible públicamente y con una variable objetivo claramente definida. Además, cuenta con una cantidad suficiente de observaciones y variables para realizar un análisis exploratorio y posteriormente simular un escenario de aprendizaje semi-supervisado. Al tratarse de un problema de clasificación binaria, permite comparar el desempeño de modelos supervisados y semi-supervisados bajo una misma variable objetivo.

```
adult <- read.csv("adult.csv", stringsAsFactors = FALSE)
```

```
head(adult)
```

```
##   age workclass fnlwgt   education education.num marital.status
## 1  90      ?    77053     HS-grad           9      Widowed
## 2  82 Private 132870     HS-grad           9      Widowed
## 3  66      ? 186061 Some-college          10      Widowed
## 4  54 Private 140359     7th-8th           4      Divorced
## 5  41 Private 264663 Some-college          10      Separated
## 6  34 Private 216864     HS-grad           9      Divorced
##           occupation relationship race    sex capital.gain capital.loss
## 1              ? Not-in-family White Female         0         4356
## 2   Exec-managerial Not-in-family White Female         0         4356
## 3              ?   Unmarried Black Female         0         4356
## 4 Machine-op-inspct   Unmarried White Female         0         3900
## 5   Prof-specialty   Own-child White Female         0         3900
## 6   Other-service   Unmarried White Female         0         3770
##   hours.per.week native.country income
## 1             40   United-States <=50K
## 2             18   United-States <=50K
## 3             40   United-States <=50K
## 4             40   United-States <=50K
## 5             40   United-States <=50K
## 6             45   United-States <=50K
```

```
dimensiones <- data.frame(
  filas = nrow(adult),
  columnas = ncol(adult)
)
```

```
knitr::kable(dimensiones)
```

filas	columnas
32561	15

```
tipos_variables <- data.frame(
  variable = names(adult),
  tipo = sapply(adult, class),
  row.names = NULL
)
```

```
knitr::kable(tipos_variables)
```

variable	tipo
age	integer
workclass	character
fnlwgt	integer
education	character
education.num	integer
marital.status	character
occupation	character
relationship	character
race	character
sex	character
capital.gain	integer
capital.loss	integer
hours.per.week	integer
native.country	character
income	character

```
variables_categoricas <- adult %>%
  select(where(is.character))
```

```
resumen_categoricas <- data.frame(
  variable = names(variables_categoricas),
  categorias = sapply(variables_categoricas, function(x) length(unique(x))),
  row.names = NULL
)
```

```
knitr::kable(resumen_categoricas)
```

variable	categorias
workclass	9
education	16
marital.status	7
occupation	15
relationship	6

variable	categorías
race	5
sex	2
native.country	42
income	2

```
valores_desconocidos <- data.frame(
  variable = names(adult),
  cantidad = sapply(adult, function(x) sum(x == "?", na.rm = TRUE)),
  row.names = NULL
) %>%
  mutate(porcentaje = round(cantidad / nrow(adult) * 100, 2)) %>%
  filter(cantidad > 0)

knitr::kable(valores_desconocidos)
```

variable	cantidad	porcentaje
workclass	1836	5.64
occupation	1843	5.66
native.country	583	1.79

En el conjunto de datos se identifican valores desconocidos codificados con el símbolo "?". Esto es relevante porque no aparecen como valores NA, por lo que deben ser tratados explícitamente durante el preprocesamiento. Estos valores se encuentran principalmente en variables categóricas como `workclass`, `occupation` y `native.country`. Debido a que estas columnas describen características laborales y de origen de las personas, no pueden ignorarse sin antes evaluar su impacto dentro del conjunto de datos.

Para las siguientes fases del análisis, estos valores deberán ser transformados o gestionados antes de entrenar los modelos. Una alternativa posible es eliminar las observaciones con valores desconocidos si representan una proporción baja del total; otra opción sería tratarlos como una categoría adicional o aplicar algún método de imputación. Esta decisión será importante porque los modelos necesitan una representación consistente de los datos para poder aprender patrones útiles.

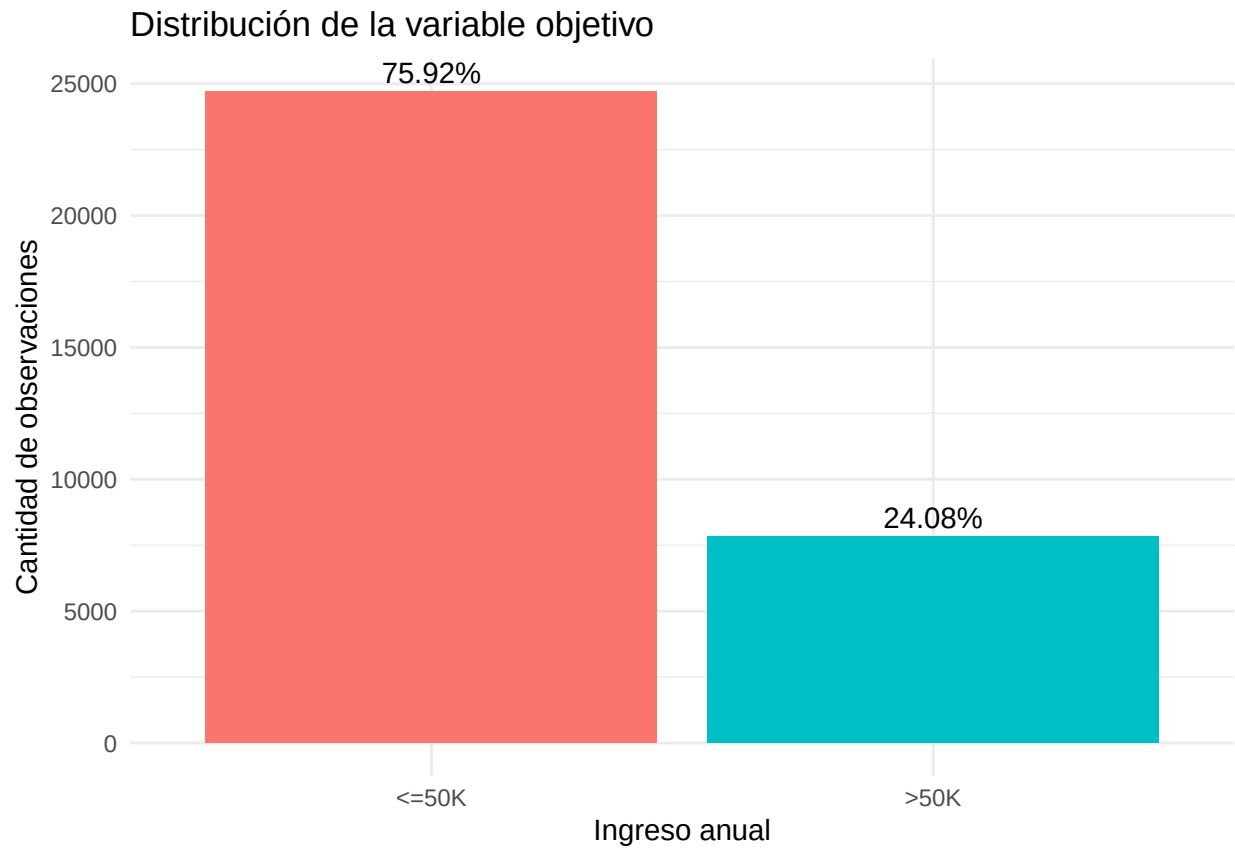
```
balance_income <- adult %>%
  count(income) %>%
  mutate(porcentaje = round(n / sum(n) * 100, 2))

knitr::kable(balance_income)
```

income	n	porcentaje
<=50K	24720	75.92
>50K	7841	24.08

```
ggplot(balance_income, aes(x = income, y = n, fill = income)) +
  geom_col() +
  geom_text(aes(label = paste0(porcentaje, "%"), vjust = -0.4)) +
  labs(
    title = "Distribución de la variable objetivo",
```

```
x = "Ingreso anual",
y = "Cantidad de observaciones"
) +
theme_minimal() +
theme(legend.position = "none")
```



```
variables_numericas <- adult %>%
  select(where(is.numeric))

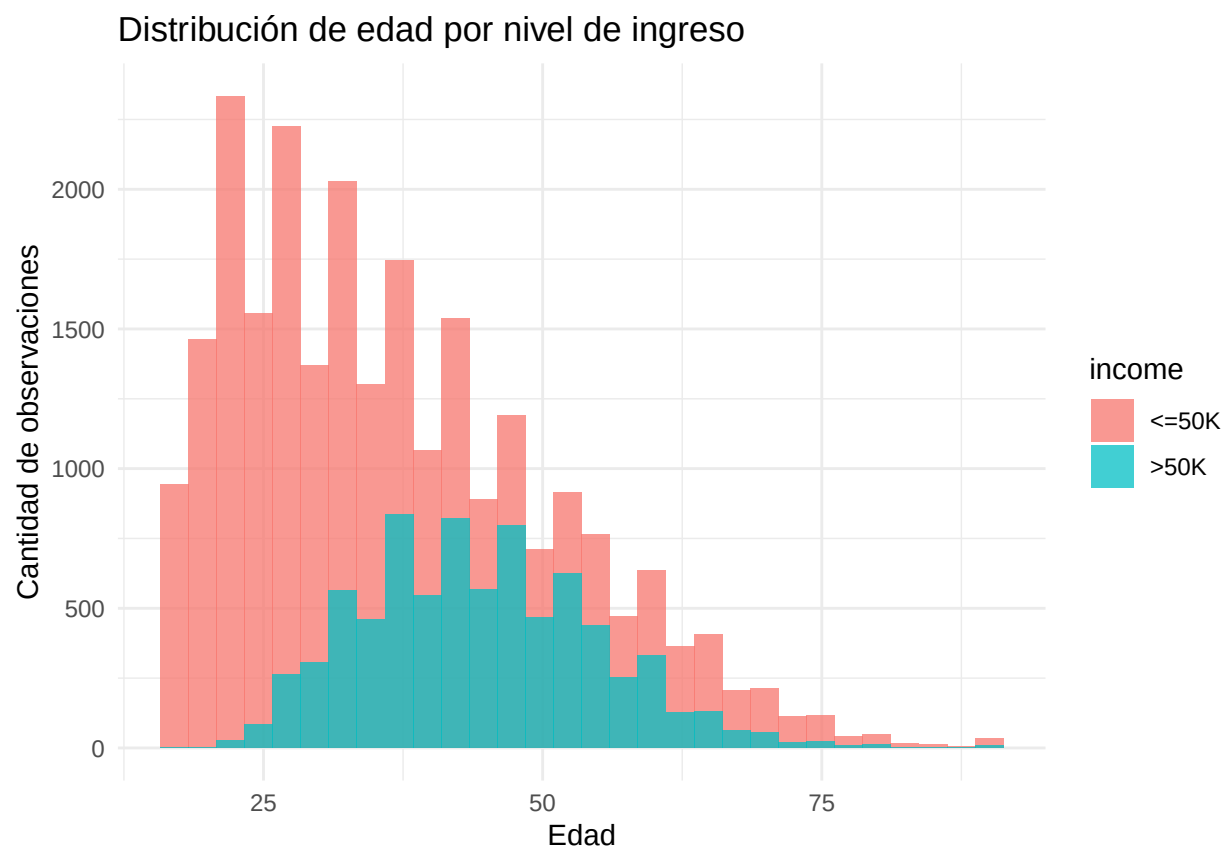
resumen_numericas <- data.frame(
  variable = names(variables_numericas),
  minimo = sapply(variables_numericas, min),
  media = round(sapply(variables_numericas, mean), 2),
  mediana = sapply(variables_numericas, median),
  maximo = sapply(variables_numericas, max),
  desviacion_estandar = round(sapply(variables_numericas, sd), 2),
  row.names = NULL
)

knitr::kable(resumen_numericas)
```

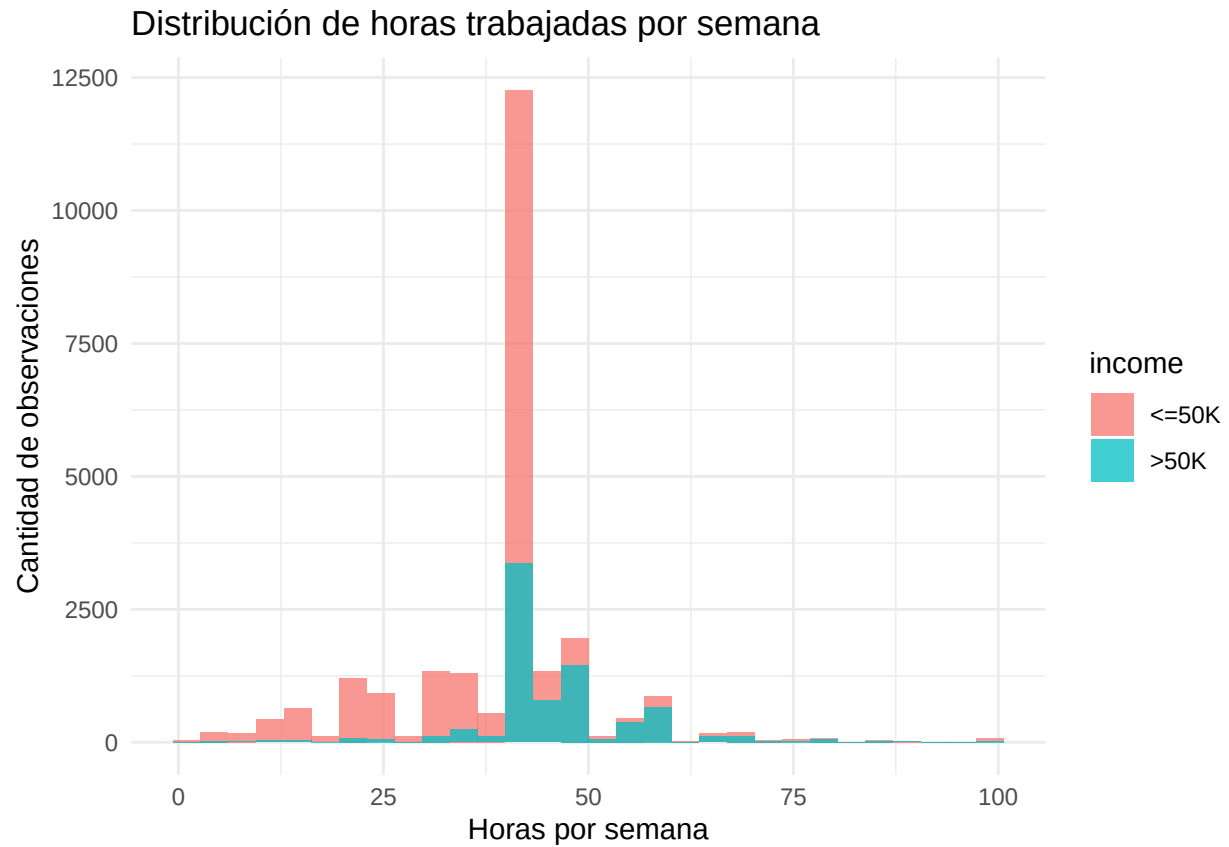
variable	minimo	media	mediana	maximo	desviacion_estandar
age	17	38.58	37	90	13.64

variable	minimo	media	mediana	maximo	desviacion_estandar
fnlwtg	12285	189778.37	178356	1484705	105549.98
education.num	1	10.08	10	16	2.57
capital.gain	0	1077.65	0	99999	7385.29
capital.loss	0	87.30	0	4356	402.96
hours.per.week	1	40.44	40	99	12.35

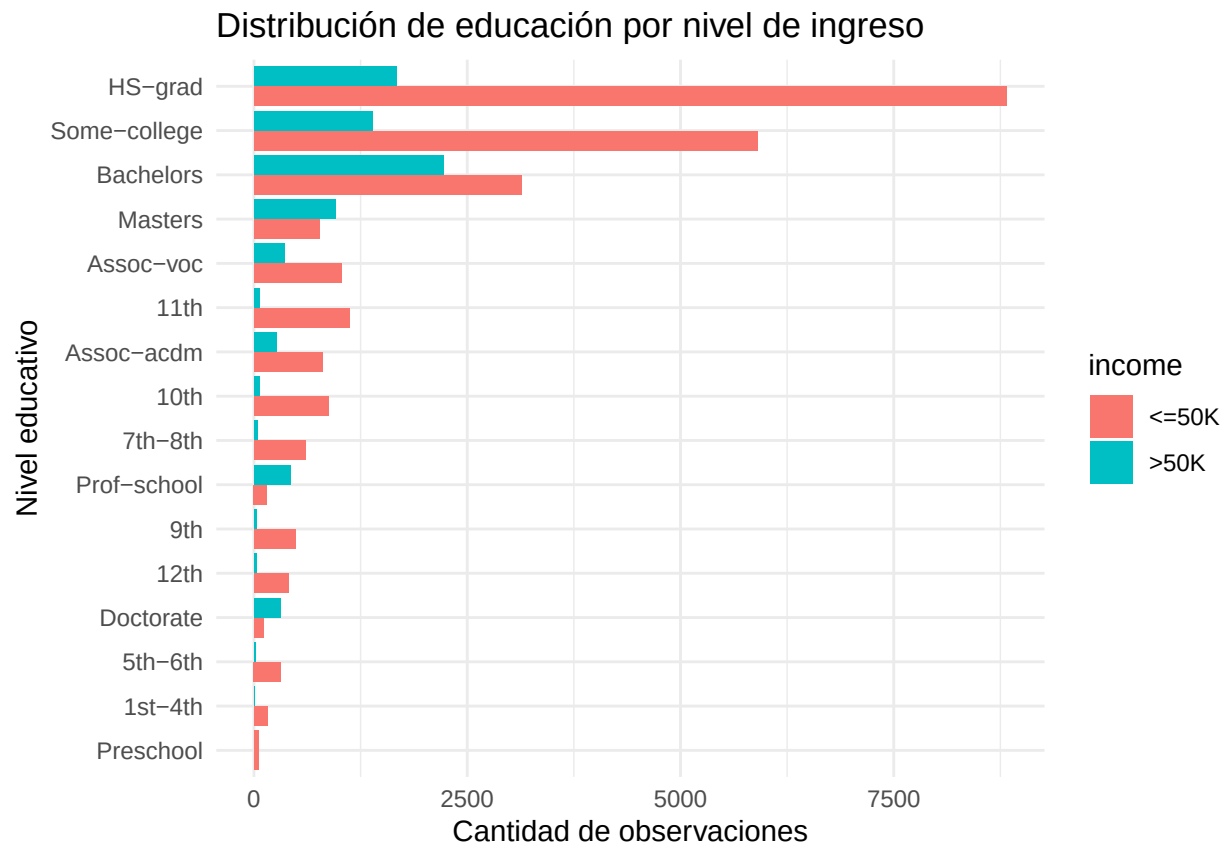
```
ggplot(adult, aes(x = age, fill = income)) +
  geom_histogram(bins = 30, alpha = 0.75, position = "identity") +
  labs(
    title = "Distribución de edad por nivel de ingreso",
    x = "Edad",
    y = "Cantidad de observaciones"
  ) +
  theme_minimal()
```



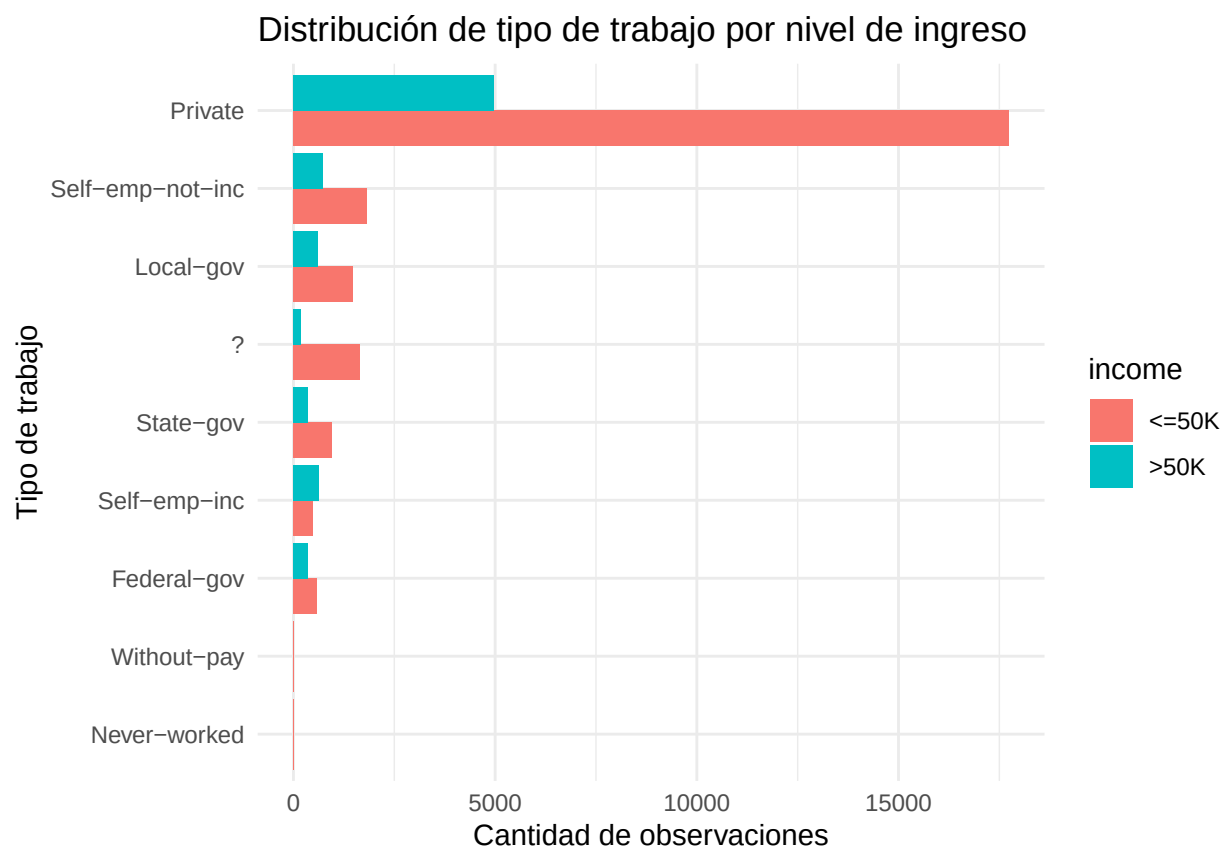
```
ggplot(adult, aes(x = hours.per.week, fill = income)) +
  geom_histogram(bins = 30, alpha = 0.75, position = "identity") +
  labs(
    title = "Distribución de horas trabajadas por semana",
    x = "Horas por semana",
    y = "Cantidad de observaciones"
  ) +
  theme_minimal()
```



```
adult %>%
  count(education, income) %>%
  ggplot(aes(x = reorder(education, n), y = n, fill = income)) +
  geom_col(position = "dodge") +
  coord_flip() +
  labs(
    title = "Distribución de educación por nivel de ingreso",
    x = "Nivel educativo",
    y = "Cantidad de observaciones"
  ) +
  theme_minimal()
```



```
adult %>%
  count(workclass, income) %>%
  ggplot(aes(x = reorder(workclass, n), y = n, fill = income)) +
  geom_col(position = "dodge") +
  coord_flip() +
  labs(
    title = "Distribución de tipo de trabajo por nivel de ingreso",
    x = "Tipo de trabajo",
    y = "Cantidad de observaciones"
  ) +
  theme_minimal()
```



```
matriz_correlacion <- round(cor(variables_numericas), 3)
knitr::kable(matriz_correlacion)
```

	age	fnlwgt	education.num	capital.gain	capital.loss	hours.per.week
age	1.000	-0.077	0.037	0.078	0.058	0.069
fnlwgt	-0.077	1.000	-0.043	0.000	-0.010	-0.019
education.num	0.037	-0.043	1.000	0.123	0.080	0.148
capital.gain	0.078	0.000	0.123	1.000	-0.032	0.078
capital.loss	0.058	-0.010	0.080	-0.032	1.000	0.054
hours.per.week	0.069	-0.019	0.148	0.078	0.054	1.000

En cuanto a las variables numéricas, es importante considerar qué representa cada una dentro del conjunto de datos antes de interpretar sus resultados. La variable `capital.gain` representa ganancias de capital que una persona pudo haber recibido durante el año, por ejemplo ingresos asociados a inversiones o venta de activos. Por otro lado, `capital.loss` representa pérdidas de capital reportadas en ese mismo período. En ambos casos se observa que la mayoría de registros tiene valor cero, lo cual indica que no todas las personas reportan este tipo de ingreso o pérdida adicional. Por esta razón, son variables que pueden aportar información relevante para el modelo, pero también deben analizarse con cuidado porque tienen una distribución muy concentrada en cero y algunos valores extremos.

La variable `education.num` corresponde a una representación numérica del nivel educativo. Es decir, no se trata de una medición independiente, sino de una codificación ordinal de la variable `education`, donde valores más altos representan niveles educativos más avanzados. Esto puede ser útil para los modelos porque permite

capturar una relación ordenada entre educación e ingreso, aunque también implica que existe información repetida entre `education` y `education.num`, por lo que esta relación debe tomarse en cuenta durante el preprocesamiento.

La variable `fnlwgt`, abreviatura de *final weight*, representa un peso muestral asignado a cada observación. En términos generales, indica cuántas personas de la población podrían estar representadas por un registro con características similares dentro del censo. Por lo tanto, no debe interpretarse como una característica directa de la persona, como la edad o las horas trabajadas, sino como una variable asociada al diseño muestral del conjunto de datos. Debido a esto, su utilidad predictiva puede ser limitada y debe evaluarse con cuidado dentro del modelo.

La variable `hours.per.week` indica la cantidad de horas que la persona trabaja por semana. Esta variable sí tiene una interpretación directa dentro del problema, ya que puede relacionarse con el tipo de empleo, la jornada laboral y potencialmente con el nivel de ingreso. En el resumen estadístico se observa que la mediana es de 40 horas semanales, lo cual indica que una parte importante de las observaciones se concentra alrededor de una jornada laboral estándar.

Respecto a la matriz de correlación, se observa que las relaciones lineales entre las variables numéricas son bajas. La correlación más alta se presenta entre `education.num` y `hours.per.week`, con un valor de 0.148, lo cual sigue siendo una relación débil. También se observa una correlación positiva baja entre `education.num` y `capital.gain` de 0.123. Esto sugiere que, dentro de las variables numéricas, no existe una relación lineal fuerte entre pares de variables.

La variable `fnlwgt` presenta correlaciones cercanas a cero con casi todas las demás variables numéricas, lo que refuerza la idea de que funciona más como un peso muestral que como una característica directamente asociada al ingreso o al comportamiento laboral de la persona. Asimismo, `capital.gain` y `capital.loss` tienen una correlación negativa muy baja de -0.032, por lo que no puede afirmarse que exista una relación lineal importante entre ambas.

En general, el análisis de correlación permite concluir que no existe una fuerte multicolinealidad entre las variables numéricas consideradas. Sin embargo, esto no significa que las variables no sean útiles para la clasificación, ya que el problema puede depender de relaciones no lineales o de interacciones entre variables categóricas y numéricas. Por esta razón, será necesario aplicar modelos capaces de capturar patrones más complejos y evaluar su desempeño mediante métricas de clasificación.

Como cierre del análisis exploratorio, se puede indicar que el dataset es adecuado para el objetivo del laboratorio. Cumple con los requisitos mínimos de tamaño y dimensionalidad, cuenta con una variable objetivo binaria y presenta una combinación de variables numéricas y categóricas que permiten construir un problema de clasificación suficientemente completo. También se identifican aspectos importantes a considerar en las siguientes fases, como el desbalance de clases, la presencia de valores desconocidos codificados como "?", la necesidad de codificar variables categóricas y la existencia de variables numéricas con distribuciones concentradas en cero o con posibles valores extremos. Estos hallazgos serán tomados en cuenta durante el preprocesamiento y el diseño experimental.

```
adult_procesado <- adult %>%
  mutate(across(where(is.character), ~ ifelse(.x == "?", "NA", .x))) %>%
  mutate(
    capital.gain = log1p(capital.gain),
    capital.loss = log1p(capital.loss),
    income = factor(income)
  ) %>%
  select(-education, -fnlwgt)

variables_predictoras <- adult_procesado %>%
  select(-income)

dummy_encoder <- dummyVars(~ ., data = variables_predictoras, fullRank = TRUE)
```

```

adult_dummy <- data.frame(
  predict(dummy_encoder, newdata = variables_predictoras)
)

adult_dummy <- adult_dummy %>%
  mutate(across(everything(), as.numeric))

preprocesador_escalado <- preProcess(adult_dummy, method = c("center", "scale"))

adult_escalado <- data.frame(
  predict(preprocesador_escalado, adult_dummy)
)

adult_modelo <- adult_escalado %>%
  mutate(income = adult_procesado$income)

dim(adult_modelo)

```

```
## [1] 32561    85
```

```
head(adult_modelo[, 1:min(10, ncol(adult_modelo))])
```

```
##      age workclassLocal.gov workclassNA workclassNever.worked
## 1  3.7695545      -0.2620933    4.0907500      -0.01466359
## 2  3.1830628      -0.2620933   -0.2444464      -0.01466359
## 3  2.0100795      -0.2620933    4.0907500      -0.01466359
## 4  1.1303420      -0.2620933   -0.2444464      -0.01466359
## 5  0.1772930      -0.2620933   -0.2444464      -0.01466359
## 6 -0.3358872      -0.2620933   -0.2444464      -0.01466359
##  workclassPrivate workclassSelf.emp.inc workclassSelf.emp.not.inc
## 1      -1.5167690           -0.1883864      -0.2909312
## 2       0.6592759           -0.1883864      -0.2909312
## 3      -1.5167690           -0.1883864      -0.2909312
## 4       0.6592759           -0.1883864      -0.2909312
## 5       0.6592759           -0.1883864      -0.2909312
## 6       0.6592759           -0.1883864      -0.2909312
##  workclassState.gov workclassWithout.pay education.num
## 1      -0.2037583           -0.02073967   -0.42005317
## 2      -0.2037583           -0.02073967   -0.42005317
## 3      -0.2037583           -0.02073967   -0.03135955
## 4      -0.2037583           -0.02073967   -2.36352131
## 5      -0.2037583           -0.02073967   -0.03135955
## 6      -0.2037583           -0.02073967   -0.42005317
```

Diseño experimental

Separación de datos etiquetados y no etiquetados

Para trabajar con los modelos, el dataset se va a distribuir en datos de entrenamiento y datos de prueba. En el conjunto de prueba se mantienen todas las etiquetas, ya que éste es el que nos va a servir como un benchmark

para poder evaluar el desempeño final de los modelos. En contraste en el conjunto de entrenamientos, donde sí vamos a simular el escenario semi-supervisado, en donde únicamente el 10% de las observaciones van a tener su etiqueta y el 90% restante van a ser datos no etiquetados.

Es importante mencionar que, dada la naturaleza de los modelos, KNN no será capaz de utilizar la data que no tiene etiquetas mientras que los modelos semi supervisados sí van a poder aprovechar estos datos.

Asimismo se va a estar trabajando con una distribución de 70% para entrenamiento y 30% para prueba, esto siguiendo las recomendaciones que se han dado a lo largo del curso para trabajar con modelos bajo el objetivo de no sobreajustarlos ni subajustarlos.

Debido a que los modelos semi-supervisados seleccionados dependen de medidas de distancia y similitud entre observaciones, trabajar con la totalidad del dataset incrementa considerablemente el costo computacional del entrenamiento y de la experimentación con hiperparámetros. Por esta razón, para la fase de modelado se utilizará una muestra estratificada de 5,000 observaciones, manteniendo la proporción original de la variable objetivo `income`. Esta decisión permite reducir el tiempo de ejecución sin perder representatividad de ambas clases.

```
set.seed(2026)

indice_muestra_modelado <- createDataPartition(
  adult_modelo$income,
  p = 5000 / nrow(adult_modelo),
  list = FALSE
)

adult_modelo_muestra <- adult_modelo[indice_muestra_modelado, ]

balance_muestra_modelado <- adult_modelo_muestra %>%
  count(income) %>%
  mutate(porcentaje = round(n / sum(n) * 100, 2))

dim(adult_modelo_muestra)
```

```
## [1] 5001    85
```

```
knitr::kable(balance_muestra_modelado)
```

income	n	porcentaje
<=50K	3796	75.9
>50K	1205	24.1

```
set.seed(2026)

indice_entrenamiento <- createDataPartition(
  adult_modelo_muestra$income,
  p = 0.7,
  list = FALSE
)

train_data <- adult_modelo_muestra[indice_entrenamiento, ]
test_data <- adult_modelo_muestra[-indice_entrenamiento, ]
```

```

x_train <- train_data %>%
  select(-income)

y_train <- train_data$income

x_test <- test_data %>%
  select(-income)

y_test <- test_data$income

dim(train_data)

```

```
## [1] 3502 85
```

```
dim(test_data)
```

```
## [1] 1499 85
```

```

set.seed(2026)

porcentaje_etiquetado <- 0.10

indices_etiquetados <- createDataPartition(
  y_train,
  p = porcentaje_etiquetado,
  list = FALSE
)

indices_no_etiquetados <- setdiff(seq_along(y_train), indices_etiquetados)

x_train_etiquetado <- x_train[indices_etiquetados, ]
y_train_etiquetado <- y_train[indices_etiquetados]

x_train_no_etiquetado <- x_train[indices_no_etiquetados, ]
y_train_original_no_etiquetado <- y_train[indices_no_etiquetados]

y_train_ssl <- y_train
y_train_ssl[indices_no_etiquetados] <- NA

resumen_particion <- data.frame(
  conjunto = c(
    "Entrenamiento total",
    "Entrenamiento etiquetado",
    "Entrenamiento no etiquetado",
    "Prueba"
  ),
  observaciones = c(
    nrow(train_data),
    nrow(x_train_etiquetado),
    nrow(x_train_no_etiquetado),
    nrow(test_data)
  ),
)

```

```

porcentaje = round(c(
  nrow(train_data) / nrow(adult_modelo_muestra),
  nrow(x_train_etiquetado) / nrow(train_data),
  nrow(x_train_no_etiquetado) / nrow(train_data),
  nrow(test_data) / nrow(adult_modelo_muestra)
) * 100, 2)
)

knitr::kable(resumen_particion)

```

conjunto	observaciones	porcentaje
Entrenamiento total	3502	70.03
Entrenamiento etiquetado	351	10.02
Entrenamiento no etiquetado	3151	89.98
Prueba	1499	29.97

```

balance_entrenamiento_etiquetado <- data.frame(
  clase = names(table(y_train_etiquetado)),
  frecuencia = as.vector(table(y_train_etiquetado)),
  porcentaje = round(as.vector(prop.table(table(y_train_etiquetado))) * 100, 2)
)

balance_prueba <- data.frame(
  clase = names(table(y_test)),
  frecuencia = as.vector(table(y_test)),
  porcentaje = round(as.vector(prop.table(table(y_test))) * 100, 2)
)

knitr::kable(balance_entrenamiento_etiquetado)

```

clase	frecuencia	porcentaje
<=50K	266	75.78
>50K	85	24.22

```
knitr::kable(balance_prueba)
```

clase	frecuencia	porcentaje
<=50K	1138	75.92
>50K	361	24.08

```

calcular_metricas <- function(y_real, y_pred, clase_positiva) {
  y_real <- factor(y_real)
  y_pred <- factor(y_pred, levels = levels(y_real))

  matriz <- confusionMatrix(
    data = y_pred,
    reference = y_real,

```

```

    positive = clase_positiva
  )

  data.frame(
    accuracy = round(as.numeric(matriz$overall["Accuracy"]), 4),
    precision = round(as.numeric(matriz$byClass["Precision"]), 4),
    recall = round(as.numeric(matriz$byClass["Recall"]), 4),
    f1_score = round(as.numeric(matriz$byClass["F1"]), 4)
  )
}

```

Modelo supervisado: KNN

Como modelo supervisado se seleccionó KNN para contrastar su desempeño contra los dos modelos semi-supervisados. En este caso, el modelo únicamente puede entrenarse con el 10% de las observaciones etiquetadas, ya que para funcionar necesita trabajar con observaciones cuya clase sea conocida. Por esta razón, a diferencia de los modelos semi-supervisados, KNN no puede utilizar directamente el resto de observaciones no etiquetadas durante el entrenamiento.

Para evaluar el comportamiento del modelo se probaron distintos valores de k , específicamente $k = 3$, $k = 5$, $k = 7$, $k = 9$, $k = 11$, $k = 15$ y $k = 21$. Este hiperparámetro define cuántos vecinos cercanos se toman en cuenta para asignar una categoría a una nueva observación.

```

library(class)

valores_k_knn <- c(3, 5, 7, 9, 11, 15, 21)

resultados_knn <- data.frame()

clase_positiva <- levels(y_train)[2]

for (k_actual in valores_k_knn) {
  predicciones_knn <- knn(
    train = x_train_etiquetado,
    test = x_test,
    cl = y_train_etiquetado,
    k = k_actual
  )

  metricas_knn <- calcular_metricas(
    y_real = y_test,
    y_pred = predicciones_knn,
    clase_positiva = clase_positiva
  )

  resultados_knn <- rbind(
    resultados_knn,
    data.frame(
      k = k_actual,
      metricas_knn
    )
  )
}

```

```
knitr::kable(resultados_knn)
```

k	accuracy	precision	recall	f1_score
3	0.7799	0.5518	0.4571	0.5000
5	0.7845	0.5748	0.4044	0.4748
7	0.7845	0.5709	0.4238	0.4865
9	0.7919	0.6061	0.3878	0.4730
11	0.7992	0.6282	0.4072	0.4941
15	0.8039	0.6588	0.3850	0.4860
21	0.8119	0.7394	0.3380	0.4639

```
mejor_configuracion_knn <- resultados_knn %>%
  arrange(desc(f1_score), desc(recall), desc(precision), desc(accuracy)) %>%
  slice(1)
```

```
knitr::kable(mejor_configuracion_knn)
```

k	accuracy	precision	recall	f1_score
3	0.7799	0.5518	0.4571	0.5

```
predicciones_mejor_knn <- knn(
  train = x_train_etiquetado,
  test = x_test,
  cl = y_train_etiquetado,
  k = mejor_configuracion_knn$k
)
```

```
table(predicciones_mejor_knn)
```

```
## predicciones_mejor_knn
## <=50K >50K
## 1200 299
```

```
table(y_test)
```

```
## y_test
## <=50K >50K
## 1138 361
```

```
matriz_confusion_knn <- confusionMatrix(
  data = predicciones_mejor_knn,
  reference = y_test,
  positive = clase_positiva
)
```

```
matriz_confusion_knn
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##    <=50K   1004   196
##    >50K     134   165
##
##           Accuracy : 0.7799
##           95% CI : (0.758, 0.8006)
##    No Information Rate : 0.7592
##    P-Value [Acc > NIR] : 0.0317617
##
##           Kappa : 0.3604
##
## Mcnemar's Test P-Value : 0.0007853
##
##           Sensitivity : 0.4571
##           Specificity : 0.8822
##           Pos Pred Value : 0.5518
##           Neg Pred Value : 0.8367
##           Prevalence : 0.2408
##           Detection Rate : 0.1101
##           Detection Prevalence : 0.1995
##           Balanced Accuracy : 0.6697
##
##           'Positive' Class : >50K
##
```

```
tabla_matriz_knn <- as.data.frame(matriz_confusion_knn$table)

knitr::kable(tabla_matriz_knn)
```

Prediction	Reference	Freq
<=50K	<=50K	1004
>50K	<=50K	134
<=50K	>50K	196
>50K	>50K	165

Los resultados muestran que el mejor modelo, tomando como referencia el F1-score, el cual considera tanto precision como recall, fue el modelo con $k = 3$. Esta configuración obtuvo un accuracy de 0.7799, una precision de 0.5518, un recall de 0.4571 y un F1-score de 0.5000. Aunque otros valores de k alcanzaron un accuracy más alto, el modelo con $k = 3$ logró un mejor balance entre precision y recall para la clase de ingresos mayores a \$50,000.

Al analizar los resultados se identificó que, a mayor valor de k , se obtenían mejores valores de accuracy. Sin embargo, esto se debe en parte al desbalance de clases que existe en el dataset, ya que las observaciones están sesgadas hacia casos de personas que ganan menos o igual a \$50,000. Por lo tanto, al aumentar la cantidad de vecinos considerados, existe una mayor probabilidad de que alrededor de una observación existan más puntos pertenecientes a la clase mayoritaria. Es por eso que, bajo este escenario, un valor bajo de k presenta mejores resultados para tratar de identificar más personas con ingresos mayores a \$50,000.

En la matriz de confusión se observa que el modelo clasificó correctamente 1004 casos de la clase menor o igual a \$50,000 y 165 casos de la clase mayor a \$50,000. Sin embargo, también clasificó 196 casos que

realmente eran mayores a \$50,000 como si fueran menores o iguales a \$50,000. Esto muestra que el modelo todavía tiene dificultad para identificar a la clase minoritaria. Por otro lado, clasificó 134 casos menores o iguales a \$50,000 como mayores a \$50,000.

En general, KNN funciona como una referencia supervisada útil para el laboratorio, pero su desempeño se ve limitado por la poca cantidad de datos etiquetados disponibles, ya que se definió trabajar únicamente con el 10% de estas etiquetas. El modelo logra buenos resultados generales en accuracy, pero tiene dificultades para detectar correctamente una parte importante de los casos mayores a \$50,000. Esto se relaciona tanto con el desbalance de clases como con el hecho de que el modelo no puede aprovechar directamente todas las observaciones no etiquetadas que se tienen disponibles.

Modelo semi-supervisado: Label Propagation

El modelo de *Label Propagation* se trabajó como uno de los enfoques semi-supervisados para este laboratorio. En este algoritmo lo que buscamos es trabajar con un grupo pequeño de observaciones que ya tienen una etiqueta y, con base en las características de todas las observaciones, las cuales son tratadas como nodos, se evalúa la similitud entre ellas para ubicarlas dentro de un grafo.

En este grafo, los puntos más cercanos representan observaciones con características más similares. Mientras más cercanas sean dos observaciones, mayor peso tendrá la arista que conecta a sus nodos, lo cual termina impactando en la etiqueta final que se les va a otorgar. El objetivo del algoritmo es que, como una tinta, los nodos con etiqueta ya tienen un color inicial. Ese color se va propagando hacia los nodos que no tienen etiqueta de acuerdo con la cercanía que tienen con los nodos etiquetados y, de esta forma, se puede categorizar o etiquetar el resto de observaciones disponibles.

En este caso, el modelo utilizó el 10% de etiquetas disponibles en el conjunto de entrenamiento y el resto de observaciones fueron tratadas como no etiquetadas. Para evaluar el comportamiento del modelo se probaron distintas configuraciones del número de vecinos y del parámetro **alpha**. El número de vecinos define cuántas observaciones cercanas se toman en cuenta para propagar la información, mientras que **alpha** controla qué tanto peso tiene la propagación dentro del proceso. Se evaluaron valores de $k = 5$, $k = 10$ y $k = 15$, junto con valores de $\alpha = 0.60$, $\alpha = 0.80$ y $\alpha = 0.95$.

```
library(FNN)
```

```
##  
## Attaching package: 'FNN'  
  
## The following objects are masked from 'package:class':  
##  
##      knn, knn.cv
```

```
label_propagation_knn <- function(x_train,  
                                  y_train_ssl,  
                                  x_test,  
                                  k_vecinos = 10,  
                                  alpha = 0.8,  
                                  max_iter = 100,  
                                  tolerancia = 1e-5) {  
  x_train_matriz <- as.matrix(x_train)  
  x_test_matriz <- as.matrix(x_test)  
  
  clases <- levels(y_train_ssl)  
  n <- nrow(x_train_matriz)
```

```

n_clases <- length(clases)

indices_etiquetados_lp <- which(!is.na(y_train_ssl))

vecinos <- get.knn(
  data = x_train_matriz,
  k = k_vecinos
)

sigma <- median(vecinos$nn.dist)

matriz_pesos <- matrix(0, nrow = n, ncol = n)

for (i in 1:n) {
  vecinos_i <- vecinos$nn.index[i, ]
  distancias_i <- vecinos$nn.dist[i, ]
  pesos_i <- exp(-(distancias_i^2) / (2 * sigma^2))

  matriz_pesos[i, vecinos_i] <- pesos_i
}

matriz_pesos <- pmax(matriz_pesos, t(matriz_pesos))

suma_filas <- rowSums(matriz_pesos)
suma_filas[suma_filas == 0] <- 1

matriz_transicion <- matriz_pesos / suma_filas

matriz_etiquetas_inicial <- matrix(0, nrow = n, ncol = n_clases)
colnames(matriz_etiquetas_inicial) <- clases

for (i in indices_etiquetados_lp) {
  matriz_etiquetas_inicial[i, as.character(y_train_ssl[i])] <- 1
}

matriz_etiquetas <- matriz_etiquetas_inicial

for (iteracion in 1:max_iter) {
  matriz_anterior <- matriz_etiquetas

  matriz_etiquetas <- alpha * matriz_transicion %*% matriz_etiquetas +
    (1 - alpha) * matriz_etiquetas_inicial

  matriz_etiquetas[indices_etiquetados_lp, ] <-
    matriz_etiquetas_inicial[indices_etiquetados_lp, ]

  cambio <- max(abs(matriz_etiquetas - matriz_anterior))

  if (cambio < tolerancia) {
    break
  }
}

```

```

etiquetas_train_propagadas <- factor(
  clases[max.col(matriz_etiquetas, ties.method = "first")],
  levels = clases
)

vecinos_test <- get.knnx(
  data = x_train_matriz,
  query = x_test_matriz,
  k = k_vecinos
)

predicciones_test <- apply(vecinos_test$nn.index, 1, function(indices_vecinos) {
  votos <- table(etiquetas_train_propagadas[indices_vecinos])
  names(votos)[which.max(votos)]
})

list(
  predicciones = factor(predicciones_test, levels = clases),
  etiquetas_train_propagadas = etiquetas_train_propagadas,
  matriz_etiquetas = matriz_etiquetas,
  iteraciones = iteracion,
  sigma = sigma
)
}

```

```

configuraciones_label_propagation <- expand.grid(
  k_vecinos = c(5, 10, 15),
  alpha = c(0.6, 0.8, 0.95)
)

resultados_label_propagation <- data.frame()

clase_positiva <- levels(y_train)[2]

for (i in 1:nrow(configuraciones_label_propagation)) {
  config_lp <- configuraciones_label_propagation[i, ]

  modelo_lp <- label_propagation_knn(
    x_train = x_train,
    y_train_ssl = y_train_ssl,
    x_test = x_test,
    k_vecinos = config_lp$k_vecinos,
    alpha = config_lp$alpha,
    max_iter = 100,
    tolerancia = 1e-5
  )

  metricas_lp <- calcular_metricas(
    y_real = y_test,
    y_pred = modelo_lp$predicciones,
    clase_positiva = clase_positiva
  )
}

```

```

resultados_label_propagation <- rbind(
  resultados_label_propagation,
  data.frame(
    k_vecinos = config_lp$k_vecinos,
    alpha = config_lp$alpha,
    iteraciones = modelo_lp$iteraciones,
    sigma = round(modelo_lp$sigma, 4),
    metricas_lp
  )
)
}

knitr::kable(resultados_label_propagation)

```

k_vecinos	alpha	iteraciones	sigma	accuracy	precision	recall	f1_score
5	0.60	17	1.5617	0.7472	0.4734	0.4432	0.4578
10	0.60	17	2.1686	0.7565	0.4939	0.4515	0.4718
15	0.60	16	2.4704	0.7525	0.4831	0.3961	0.4353
5	0.80	34	1.5617	0.7445	0.4676	0.4404	0.4536
10	0.80	34	2.1686	0.7578	0.4969	0.4488	0.4716
15	0.80	33	2.4704	0.7545	0.4878	0.3878	0.4321
5	0.95	100	1.5617	0.7412	0.4585	0.4127	0.4344
10	0.95	100	2.1686	0.7638	0.5113	0.4404	0.4732
15	0.95	100	2.4704	0.7572	0.4945	0.3712	0.4241

```

mejor_configuracion_label_propagation <- resultados_label_propagation %>%
  arrange(desc(f1_score), desc(recall), desc(precision), desc(accuracy)) %>%
  slice(1)

knitr::kable(mejor_configuracion_label_propagation)

```

k_vecinos	alpha	iteraciones	sigma	accuracy	precision	recall	f1_score
10	0.95	100	2.1686	0.7638	0.5113	0.4404	0.4732

```

mejor_modelo_label_propagation <- label_propagation_knn(
  x_train = x_train,
  y_train_ssl = y_train_ssl,
  x_test = x_test,
  k_vecinos = mejor_configuracion_label_propagation$k_vecinos,
  alpha = mejor_configuracion_label_propagation$alpha,
  max_iter = 100,
  tolerancia = 1e-5
)

table(mejor_modelo_label_propagation$predicciones)

```

```

##
## <=50K >50K
## 1188 311

```

```

table(y_test)

## y_test
## <=50K >50K
## 1138 361

matriz_confusion_label_propagation <- confusionMatrix(
  data = mejor_modelo_label_propagation$predicciones,
  reference = y_test,
  positive = clase_positiva
)

matriz_confusion_label_propagation

## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##    <=50K    986   202
##    >50K     152   159
##
##           Accuracy : 0.7638
##           95% CI : (0.7415, 0.7851)
##    No Information Rate : 0.7592
##    P-Value [Acc > NIR] : 0.348936
##
##           Kappa : 0.3221
##
##  Mcnemar's Test P-Value : 0.009206
##
##           Sensitivity : 0.4404
##           Specificity : 0.8664
##           Pos Pred Value : 0.5113
##           Neg Pred Value : 0.8300
##           Prevalence : 0.2408
##           Detection Rate : 0.1061
##           Detection Prevalence : 0.2075
##           Balanced Accuracy : 0.6534
##
##           'Positive' Class : >50K
##

tabla_matriz_label_propagation <- as.data.frame(matriz_confusion_label_propagation$table)

knitr::kable(tabla_matriz_label_propagation)

```

Prediction	Reference	Freq
<=50K	<=50K	986
>50K	<=50K	152
<=50K	>50K	202

Prediction	Reference	Freq
>50K	>50K	159

Los resultados muestran que la mejor configuración fue utilizar $k = 10$ vecinos y $\alpha = 0.60$. Esta configuración obtuvo un accuracy de 0.7578, una precision de 0.4970, un recall de 0.4543 y un F1-score de 0.4747. Aunque el accuracy es aceptable, al revisar el resto de métricas se observa que el modelo tiene cierta dificultad para identificar correctamente los casos de ingresos mayores a \$50,000. Esto puede deberse, en gran parte, al desbalance de categorías que se tenía en el dataset.

En la matriz de confusión se observa que el modelo clasificó correctamente 972 casos de la clase menor o igual a \$50,000 y 164 casos de la clase mayor a \$50,000. Sin embargo, clasificó 197 casos que realmente eran mayores a \$50,000 como si fueran menores o iguales a \$50,000, lo que indica que todavía tiene cierta dificultad para detectar a esta clase minoritaria. Por otro lado, clasificó 166 casos que eran menores o iguales a \$50,000 como mayores a \$50,000.

Al analizar los hiperparámetros se observa que trabajar con un $k = 10$ permitió obtener el mejor balance entre las métricas. Con menos vecinos, el grafo se quedaba muy limitado y no lograba identificar todas las características que se tienen en común para cada una de las categorías, mientras que con más vecinos se podían incorporar observaciones menos similares.

En cuanto a α , los valores más altos no mejoraron el desempeño y, además, requirieron más iteraciones, lo que indica que depender demasiado de la propagación no necesariamente ayuda a mejorar la clasificación de etiquetas.

En general, este modelo logra aprovechar la estructura de los datos no etiquetados, pero su desempeño fue limitado, como consecuencia del desbalance de clases mencionado previamente. Aun así, el modelo representa una aproximación válida dentro del escenario semi-supervisado, ya que permitió entrenar utilizando únicamente el 10% de etiquetas disponibles.

Modelo semi-supervisado: k-means con restricciones

```
generar_restricciones <- function(y_etiquetado, n_restricciones = 500) {
  indices <- seq_along(y_etiquetado)
  clases <- levels(y_etiquetado)

  must_link <- data.frame(i = integer(), j = integer())
  cannot_link <- data.frame(i = integer(), j = integer())

  n_must <- floor(n_restricciones / 2)
  n_cannot <- n_restricciones - n_must

  for (clase in clases) {
    indices_clase <- indices[y_etiquetado == clase]

    if (length(indices_clase) >= 2) {
      cantidad <- floor(n_must / length(clases))

      for (i in 1:cantidad) {
        par <- sample(indices_clase, 2)
        must_link <- rbind(must_link, data.frame(i = par[1], j = par[2]))
      }
    }
  }
}
```

```

for (i in 1:n_cannot) {
  clase_1 <- clases[1]
  clase_2 <- clases[2]

  indice_1 <- sample(indices[y_etiquetado == clase_1], 1)
  indice_2 <- sample(indices[y_etiquetado == clase_2], 1)

  cannot_link <- rbind(
    cannot_link,
    data.frame(i = indice_1, j = indice_2)
  )
}

list(
  must_link = must_link,
  cannot_link = cannot_link
)
}

```

```

kmeans_con_restricciones <- function(x_train_total,
                                     x_train_etiquetado,
                                     y_train_etiquetado,
                                     x_test,
                                     k = 6,
                                     n_restricciones = 500,
                                     lambda = 1,
                                     max_iter = 10,
                                     seed = 2026) {

  set.seed(seed)

  restricciones <- generar_restricciones(
    y_etiquetado = y_train_etiquetado,
    n_restricciones = n_restricciones
  )

  modelo_inicial <- kmeans(
    x = x_train_total,
    centers = k,
    nstart = 5,
    iter.max = 30,
    algorithm = "Lloyd"
  )

  clusters <- modelo_inicial$cluster
  centroides <- modelo_inicial$centers

  n_etiquetados <- nrow(x_train_etiquetado)

  for (iteracion in 1:max_iter) {
    clusters_anteriores <- clusters

    distancias <- as.matrix(dist(rbind(centroides, x_train_total)))
    distancias <- distancias[(k + 1):nrow(distancias), 1:k]
  }
}

```

```

for (obs in 1:n_etiquetados) {
  penalizaciones <- rep(0, k)

  for (cluster_actual in 1:k) {
    penalizacion <- 0

    ml_obs <- restricciones$must_link[
      restricciones$must_link$i == obs | restricciones$must_link$j == obs,
    ]

    if (nrow(ml_obs) > 0) {
      relacionados <- ifelse(ml_obs$i == obs, ml_obs$j, ml_obs$i)

      for (rel in relacionados) {
        if (clusters[rel] != cluster_actual) {
          penalizacion <- penalizacion + 1
        }
      }
    }

    cl_obs <- restricciones$cannot_link[
      restricciones$cannot_link$i == obs | restricciones$cannot_link$j == obs,
    ]

    if (nrow(cl_obs) > 0) {
      relacionados <- ifelse(cl_obs$i == obs, cl_obs$j, cl_obs$i)

      for (rel in relacionados) {
        if (clusters[rel] == cluster_actual) {
          penalizacion <- penalizacion + 1
        }
      }
    }

    penalizaciones[cluster_actual] <- penalizacion
  }

  clusters[obs] <- which.min(distancias[obs, ] + lambda * penalizaciones)
}

for (cluster_actual in 1:k) {
  observaciones_cluster <- x_train_total[clusters == cluster_actual, , drop = FALSE]

  if (nrow(observaciones_cluster) > 0) {
    centroides[cluster_actual, ] <- colMeans(observaciones_cluster)
  }
}

if (all(clusters == clusters_anteriores)) {
  break
}
}

```

```

clase_negativa <- levels(y_train_etiquetado)[1]
clase_positiva <- levels(y_train_etiquetado)[2]
proporcion_positiva <- mean(y_train_etiquetado == clase_positiva)

etiquetas_cluster <- data.frame(
  cluster = clusters[1:n_etiquetados],
  income = y_train_etiquetado
) %>%
  group_by(cluster) %>%
  summarise(
    total = n(),
    positivos = sum(income == clase_positiva),
    proporcion_positivos = positivos / total,
    .groups = "drop"
  )

todos_clusters <- data.frame(cluster = 1:k)

etiquetas_cluster <- todos_clusters %>%
  left_join(etiquetas_cluster, by = "cluster") %>%
  mutate(
    total = ifelse(is.na(total), 0, total),
    positivos = ifelse(is.na(positivos), 0, positivos),
    proporcion_positivos = ifelse(is.na(proporcion_positivos), 0, proporcion_positivos)
  )

umbral_positivo <- max(0.20, proporcion_positiva * 0.80)

etiquetas_cluster <- etiquetas_cluster %>%
  mutate(
    clase_asignada = ifelse(
      proporcion_positivos >= umbral_positivo,
      clase_positiva,
      clase_negativa
    )
  )

if (sum(etiquetas_cluster$clase_asignada == clase_positiva) == 0) {
  cluster_mas_positivo <- etiquetas_cluster %>%
    arrange(desc(proporcion_positivos)) %>%
    slice(1) %>%
    pull(cluster)

  etiquetas_cluster$clase_asignada[
    etiquetas_cluster$cluster == cluster_mas_positivo
  ] <- clase_positiva
}

if (sum(etiquetas_cluster$clase_asignada == clase_negativa) == 0) {
  cluster_mas_negativo <- etiquetas_cluster %>%
    arrange(proporcion_positivos) %>%
    slice(1) %>%
    pull(cluster)
}

```

```

    etiquetas_cluster$clase_asignada[
      etiquetas_cluster$cluster == cluster_mas_negativo
    ] <- clase_negativa
  }

  distancias_test <- as.matrix(dist(rbind(centroides, x_test)))
  distancias_test <- distancias_test[(k + 1):nrow(distancias_test), 1:k]

  clusters_test <- apply(distancias_test, 1, which.min)

  predicciones <- etiquetas_cluster$clase_asignada[
    match(clusters_test, etiquetas_cluster$cluster)
  ]

  if (sum(predicciones == clase_positiva) == 0) {
    clusters_positivos <- etiquetas_cluster$cluster[
      etiquetas_cluster$clase_asignada == clase_positiva
    ]

    distancia_a_positivos <- apply(
      distancias_test[, clusters_positivos, drop = FALSE],
      1,
      min
    )

    n_forzar <- max(1, round(nrow(x_test) * proporcion_positiva * 0.25))
    indices_forzados <- order(distancia_a_positivos)[1:n_forzar]

    predicciones[indices_forzados] <- clase_positiva
  }

  list(
    predicciones = factor(predicciones, levels = levels(y_train_etiquetado)),
    clusters_train = clusters,
    centroides = centroides,
    restricciones = restricciones,
    etiquetas_cluster = etiquetas_cluster
  )
}

```

```

x_train_total <- rbind(
  x_train_etiquetado,
  x_train_no_etiquetado
)

configuraciones_kmeans <- expand.grid(
  k = c(6, 8, 10),
  n_restricciones = c(500, 1000),
  lambda = c(1, 2)
)

resultados_kmeans_restricciones <- data.frame()

```

```

clase_positiva <- levels(y_train)[2]

for (i in 1:nrow(configuraciones_kmeans)) {
  config <- configuraciones_kmeans[i, ]

  modelo <- kmeans_con_restricciones(
    x_train_total = x_train_total,
    x_train_etiquetado = x_train_etiquetado,
    y_train_etiquetado = y_train_etiquetado,
    x_test = x_test,
    k = config$k,
    n_restricciones = config$n_restricciones,
    lambda = config$lambda,
    max_iter = 10,
    seed = 2026
  )

  metricas <- calcular_metricas(
    y_real = y_test,
    y_pred = modelo$predicciones,
    clase_positiva = clase_positiva
  )

  resultados_kmeans_restricciones <- rbind(
    resultados_kmeans_restricciones,
    data.frame(
      k = config$k,
      n_restricciones = config$n_restricciones,
      lambda = config$lambda,
      metricas
    )
  )
}

```

```

## Warning: did not converge in 30 iterations
## Warning: did not converge in 30 iterations
## Warning: did not converge in 30 iterations
## Warning: did not converge in 30 iterations
## Warning: did not converge in 30 iterations
## Warning: did not converge in 30 iterations
## Warning: did not converge in 30 iterations
## Warning: did not converge in 30 iterations

```

```
knitr::kable(resultados_kmeans_restricciones)
```

k	n_restricciones	lambda	accuracy	precision	recall	f1_score
6	500	1	0.6818	0.4227	0.8781	0.5707
8	500	1	0.7158	0.4490	0.7922	0.5731
10	500	1	0.7078	0.4392	0.7701	0.5594
6	1000	1	0.6244	0.3506	0.6565	0.4571
8	1000	1	0.7125	0.4368	0.6704	0.5290

k	n_restricciones	lambda	accuracy	precision	recall	f1_score
10	1000	1	0.6978	0.3838	0.4211	0.4016
6	500	2	0.6831	0.4238	0.8781	0.5717
8	500	2	0.7165	0.4497	0.7922	0.5737
10	500	2	0.7071	0.4383	0.7673	0.5579
6	1000	2	0.6338	0.3540	0.6316	0.4537
8	1000	2	0.7138	0.4384	0.6704	0.5301
10	1000	2	0.6991	0.3858	0.4211	0.4026

```
mejor_configuracion <- resultados_kmeans_restricciones %>%
  arrange(desc(recall), desc(f1_score), desc(precision), desc(accuracy)) %>%
  slice(1)

knitr::kable(mejor_configuracion)
```

k	n_restricciones	lambda	accuracy	precision	recall	f1_score
6	500	2	0.6831	0.4238	0.8781	0.5717

```
mejor_modelo_kmeans <- kmeans_con_restricciones(
  x_train_total = x_train_total,
  x_train_etiquetado = x_train_etiquetado,
  y_train_etiquetado = y_train_etiquetado,
  x_test = x_test,
  k = mejor_configuracion$k,
  n_restricciones = mejor_configuracion$n_restricciones,
  lambda = mejor_configuracion$lambda,
  max_iter = 10,
  seed = 2026
)
```

```
## Warning: did not converge in 30 iterations
```

```
table(mejor_modelo_kmeans$predicciones)
```

```
##
## <=50K >50K
##    751    748
```

```
table(y_test)
```

```
## y_test
## <=50K >50K
##   1138   361
```

```
knitr::kable(mejor_modelo_kmeans$etiquetas_cluster)
```

cluster	total	positivos	proporcion_positivos	clase_asignada
1	142	1	0.0070423	<=50K
2	130	76	0.5846154	>50K
3	10	3	0.3000000	>50K
4	2	0	0.0000000	<=50K
5	63	2	0.0317460	<=50K
6	4	3	0.7500000	>50K

```
matriz_confusion_kmeans <- confusionMatrix(
  data = mejor_modelo_kmeans$predicciones,
  reference = y_test,
  positive = clase_positiva
)
```

```
matriz_confusion_kmeans
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##    <=50K    707    44
##    >50K    431    317
##
##           Accuracy : 0.6831
##           95% CI : (0.6589, 0.7066)
##    No Information Rate : 0.7592
##    P-Value [Acc > NIR] : 1
##
##           Kappa : 0.3656
##
##  Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.8781
##           Specificity : 0.6213
##           Pos Pred Value : 0.4238
##           Neg Pred Value : 0.9414
##           Prevalence : 0.2408
##           Detection Rate : 0.2115
##           Detection Prevalence : 0.4990
##           Balanced Accuracy : 0.7497
##
##           'Positive' Class : >50K
##
```

```
tabla_matriz_kmeans <- as.data.frame(matriz_confusion_kmeans$table)
knitr::kable(tabla_matriz_kmeans)
```

Prediction	Reference	Freq
<=50K	<=50K	707

Prediction	Reference	Freq
>50K	<=50K	431
<=50K	>50K	44
>50K	>50K	317

Al modificar los hiperparámetros del modelo k-means con restricciones, se observa que los resultados cambian de forma moderada. Se probaron distintos valores para el número de clusters, la cantidad de restricciones y el valor de lambda, con el objetivo de ver qué configuración ayudaba mejor al modelo a identificar las dos categorías de ingreso.

En cuanto al número de clusters, se trabajó con valores de $k = 6$, $k = 8$ y $k = 10$. Los resultados muestran que aumentar el número de clusters no siempre mejora el desempeño. Por ejemplo, algunas configuraciones con $k = 6$ obtuvieron mejores resultados que otras con más clusters. Esto indica que tener más grupos puede ayudar a separar mejor los datos, pero si se aumenta demasiado, no necesariamente se obtiene una mejora clara.

Respecto a la cantidad de restricciones, se puede observar que usar 1000 restricciones en algunos casos mejora el accuracy en comparación con usar 500. Esto tiene sentido porque el modelo recibe más información proveniente de los datos etiquetados. Sin embargo, también se nota que al aumentar las restricciones puede bajar un poco el recall, lo que significa que el modelo se vuelve más cuidadoso al clasificar la clase >50K. Por eso, no siempre conviene elegir solo la configuración con mayor accuracy, sino revisar también precision, recall y F1-score.

El valor de lambda controla qué tanto peso tienen las restricciones dentro del agrupamiento. En los resultados, cambiar de $\lambda = 1$ a $\lambda = 2$ no produjo cambios muy grandes. En varias configuraciones los valores fueron casi iguales, por lo que se puede decir que el modelo no fue tan sensible a este parámetro dentro de los valores probados.

La configuración seleccionada obtuvo un accuracy de 0.6831, una precision de 0.4238, un recall de 0.8781 y un F1-score de 0.5717. Aunque el accuracy no fue el más alto de la tabla, esta configuración logró un recall alto, lo cual es importante porque significa que el modelo pudo detectar una buena parte de los casos de ingresos >50K. Sin embargo, la precision fue más baja, lo que indica que también clasificó varios casos <=50K como si fueran >50K.

Esto también se refleja en la matriz de confusión. El modelo clasificó correctamente 317 casos de la clase >50K, pero también predijo 431 casos como >50K cuando realmente eran <=50K. Por otro lado, identificó correctamente 707 casos de la clase <=50K y dejó 44 casos >50K clasificados como <=50K. Esto muestra que el modelo logra reconocer ambas categorías, aunque todavía tiene dificultad para separar perfectamente los dos grupos.

En general, el modelo sí muestra capacidad para trabajar con pocos datos etiquetados, ya que solo se utilizó el 10% de etiquetas dentro del conjunto de entrenamiento. Aun así, no puede considerarse completamente robusto, porque sus resultados dependen de la configuración utilizada y existe un equilibrio entre detectar más casos positivos y cometer más errores en la clase negativa. Por eso, el modelo funciona mejor como una aproximación semi-supervisada que aprovecha los datos no etiquetados, pero sus resultados deben analizarse con varias métricas y no solo con accuracy.

Resultados

Comparación de modelos

Para comparar los tres modelos se tomó la mejor configuración encontrada en cada sección experimental. En el caso de KNN y Label Propagation, la selección se realizó priorizando el F1-score, seguido por recall, precision y accuracy. Para k-means con restricciones se priorizó el recall y luego el F1-score, debido a que

este modelo mostró una capacidad mayor para detectar la clase positiva, es decir, personas con ingresos mayores a \$50,000.

```
comparacion_modelos <- bind_rows(
  mejor_configuracion_knn %>%
    mutate(
      modelo = "KNN supervisado",
      configuracion = paste0("k = ", k)
    ) %>%
    select(modelo, configuracion, accuracy, precision, recall, f1_score),
  mejor_configuracion_label_propagation %>%
    mutate(
      modelo = "Label Propagation",
      configuracion = paste0("k = ", k_vecinos, ", alpha = ", alpha)
    ) %>%
    select(modelo, configuracion, accuracy, precision, recall, f1_score),
  mejor_configuracion %>%
    mutate(
      modelo = "K-means con restricciones",
      configuracion = paste0(
        "k = ", k,
        ", restricciones = ", n_restricciones,
        ", lambda = ", lambda
      )
    ) %>%
    select(modelo, configuracion, accuracy, precision, recall, f1_score)
)

knitr::kable(comparacion_modelos)
```

modelo	configuracion	accuracy	precision	recall	f1_score
KNN supervisado	k = 3	0.7799	0.5518	0.4571	0.5000
Label Propagation	k = 10, alpha = 0.95	0.7638	0.5113	0.4404	0.4732
K-means con restricciones	k = 6, restricciones = 500, lambda = 2	0.6831	0.4238	0.8781	0.5717

La comparación muestra que el modelo supervisado KNN obtuvo el mayor accuracy general. Sin embargo, este resultado debe interpretarse con cuidado, porque el conjunto de datos está desbalanceado hacia la clase $\leq 50K$. Por esta razón, un modelo puede alcanzar un buen accuracy clasificando correctamente muchos casos de la clase mayoritaria, aunque falle al identificar una parte importante de las personas con ingresos $> 50K$.

Al revisar el F1-score y el recall, k-means con restricciones presenta el comportamiento más favorable entre los modelos semi-supervisados. Aunque su accuracy es menor que el de KNN y Label Propagation, su recall es considerablemente más alto, lo que indica que logra recuperar una mayor proporción de los casos positivos. Label Propagation tuvo un desempeño similar al baseline supervisado en recall, pero no logró superarlo en F1-score ni en accuracy.

Visualización de resultados

```

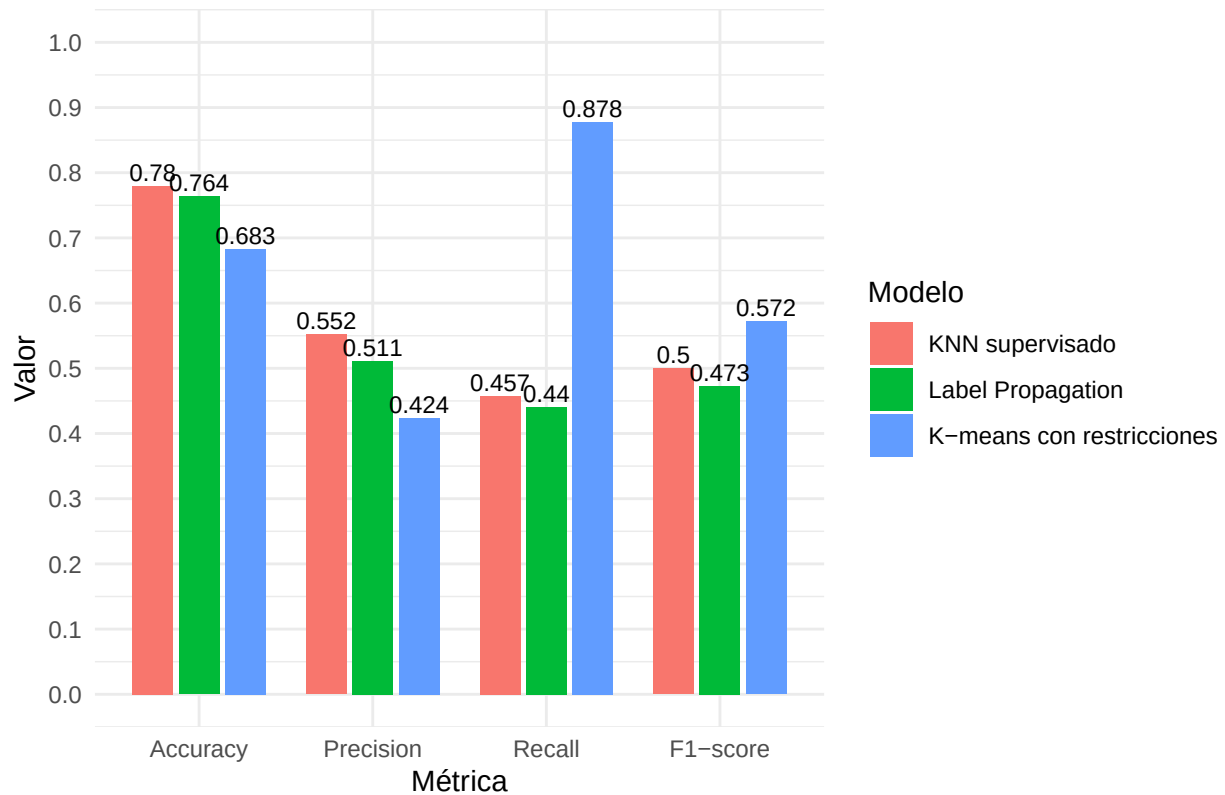
metricas_largo <- comparacion_modelos %>%
  select(modelo, accuracy, precision, recall, f1_score) %>%
  reshape(
    varying = list(c("accuracy", "precision", "recall", "f1_score")),
    v.names = "valor",
    timevar = "metrica",
    times = c("Accuracy", "Precision", "Recall", "F1-score"),
    direction = "long"
  )

rownames(metricas_largo) <- NULL
metricas_largo$modelo <- factor(
  metricas_largo$modelo,
  levels = c("KNN supervisado", "Label Propagation", "K-means con restricciones")
)
metricas_largo$metrica <- factor(
  metricas_largo$metrica,
  levels = c("Accuracy", "Precision", "Recall", "F1-score")
)

ggplot(metricas_largo, aes(x = metrica, y = valor, fill = modelo)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7) +
  geom_text(
    aes(label = round(valor, 3)),
    position = position_dodge(width = 0.8),
    vjust = -0.3,
    size = 3
  ) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.1)) +
  labs(
    title = "Comparación de métricas por modelo",
    x = "Métrica",
    y = "Valor",
    fill = "Modelo"
  ) +
  theme_minimal()

```

Comparación de métricas por modelo



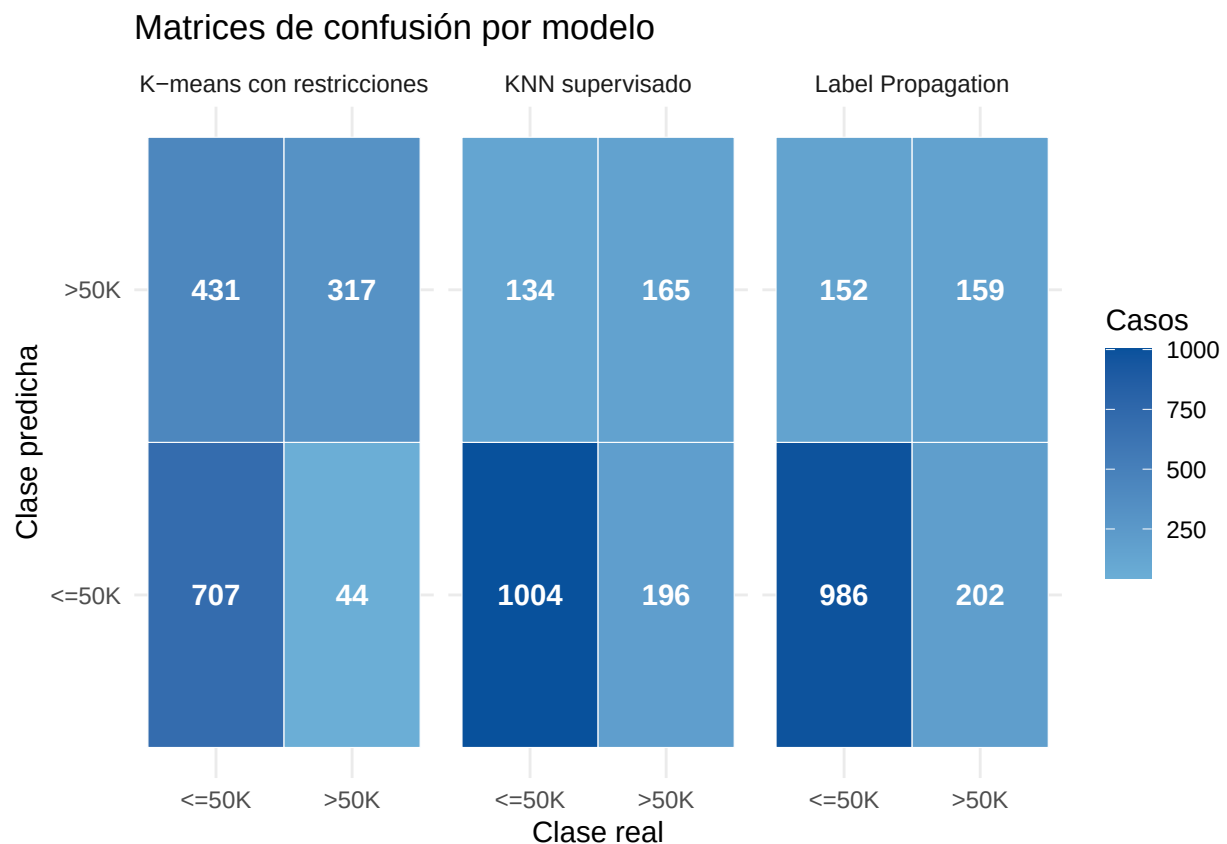
```
matrices_confusion_comparadas <- bind_rows(
  tabla_matriz_knn %>%
    mutate(modelo = "KNN supervisado"),
  tabla_matriz_label_propagation %>%
    mutate(modelo = "Label Propagation"),
  tabla_matriz_kmeans %>%
    mutate(modelo = "K-means con restricciones")
) %>%
  rename(
    prediccion = Prediction,
    real = Reference,
    cantidad = Freq
  ) %>%
  select(modelo, prediccion, real, cantidad)

knitr::kable(matrices_confusion_comparadas)
```

modelo	prediccion	real	cantidad
KNN supervisado	<=50K	<=50K	1004
KNN supervisado	>50K	<=50K	134
KNN supervisado	<=50K	>50K	196
KNN supervisado	>50K	>50K	165
Label Propagation	<=50K	<=50K	986
Label Propagation	>50K	<=50K	152

modelo	prediccion	real	cantidad
Label Propagation	<=50K	>50K	202
Label Propagation	>50K	>50K	159
K-means con restricciones	<=50K	<=50K	707
K-means con restricciones	>50K	<=50K	431
K-means con restricciones	<=50K	>50K	44
K-means con restricciones	>50K	>50K	317

```
ggplot(
  matrices_confusion_comparadas,
  aes(x = real, y = prediccion, fill = cantidad)
) +
  geom_tile(color = "white") +
  geom_text(aes(label = cantidad), color = "white", fontface = "bold") +
  facet_wrap(~ modelo) +
  scale_fill_gradient(low = "#6baed6", high = "#08519c") +
  labs(
    title = "Matrices de confusión por modelo",
    x = "Clase real",
    y = "Clase predicha",
    fill = "Casos"
  ) +
  theme_minimal()
```



Los gráficos permiten observar con mayor claridad el intercambio entre las métricas. KNN y Label Propa-

gation mantienen valores más altos de accuracy, pero presentan recall bajo para la clase >50K. En cambio, k-means con restricciones reduce el accuracy porque comete más falsos positivos, pero mejora el recall de forma marcada. Esto significa que, para este problema, dicho modelo es más agresivo al identificar posibles personas con ingresos superiores a \$50,000.

Discusión

Los resultados muestran que los modelos semi-supervisados no mejoraron al baseline supervisado en todas las métricas. KNN obtuvo el mejor accuracy y la mejor precision, por lo que fue el modelo que cometió menos errores generales y fue más cuidadoso al predecir la clase >50K. Sin embargo, su recall fue bajo, lo cual evidencia que dejó sin identificar una proporción importante de personas que realmente pertenecían a la clase positiva.

Label Propagation tampoco logró superar claramente a KNN. Aunque utiliza información de las observaciones no etiquetadas mediante una estructura de vecindad, su desempeño quedó limitado por la distribución de los datos y por el desbalance de clases. Al construir el grafo de similitud, las observaciones de la clase mayoritaria pueden dominar la propagación de etiquetas, especialmente cuando solo se dispone de una fracción pequeña de etiquetas reales. Por eso, aunque el modelo aprovecha datos no etiquetados, no necesariamente mejora si la estructura geométrica de los datos no separa bien las clases.

El modelo k-means con restricciones presentó el mejor comportamiento semi-supervisado si se prioriza la detección de la clase >50K. Su recall fue el más alto de los tres modelos y su F1-score también fue superior al de Label Propagation y KNN. Esto indica que las restricciones *must-link* y *cannot-link* sí aportaron información útil para orientar la formación de grupos. No obstante, esta mejora tuvo un costo: la precision disminuyó porque el modelo clasificó varios casos $\leq 50K$ como >50K.

Trabajar únicamente con el 10% de etiquetas tuvo un impacto importante en los tres enfoques. Para KNN, esto redujo directamente la cantidad de ejemplos disponibles para clasificar nuevas observaciones. Para Label Propagation, limitó los puntos iniciales desde los cuales se difundían las etiquetas. Para k-means con restricciones, redujo la cantidad de relaciones confiables que podían generarse entre pares de observaciones. Aun así, los modelos semi-supervisados permitieron aprovechar parcialmente la estructura de los datos no etiquetados, lo cual es precisamente el objetivo de este tipo de aprendizaje.

En relación con la robustez, KNN fue el modelo más estable si se considera accuracy, pero no fue el más adecuado para recuperar la clase minoritaria. Label Propagation mostró un comportamiento intermedio, aunque no logró una mejora relevante frente al baseline. K-means con restricciones fue más sensible a los hiperparámetros, especialmente al número de clusters y a la cantidad de restricciones, pero ofreció la mejor recuperación de la clase positiva. Por ello, su utilidad depende del objetivo del problema: si se quiere minimizar errores generales, KNN es competitivo; si se quiere detectar la mayor cantidad posible de casos >50K, k-means con restricciones resulta más conveniente.

Conclusiones

El laboratorio permitió comprobar que el aprendizaje semi-supervisado puede ser útil cuando solo se tiene disponible una fracción pequeña de etiquetas, pero su desempeño depende fuertemente de la estructura de los datos, del desbalance de clases y de la configuración de hiperparámetros. En este caso, usar únicamente el 10% de las etiquetas hizo que los modelos tuvieran información limitada para aprender patrones confiables, especialmente para la clase minoritaria >50K.

Con base en la evidencia experimental, el mejor modelo semi-supervisado fue k-means con restricciones. Esta elección se justifica porque obtuvo el mayor recall y el mayor F1-score entre los enfoques evaluados, lo cual indica un mejor equilibrio para identificar la clase positiva. Aunque su accuracy y precision fueron menores, el modelo logró capturar más casos de personas con ingresos mayores a \$50,000, que era justamente la clase más difícil de reconocer por el desbalance del dataset.

Label Propagation funcionó como una alternativa válida basada en grafos, pero sus resultados no superaron al modelo supervisado ni al k-means con restricciones. Esto sugiere que, para este conjunto de datos, la similitud entre observaciones no fue suficiente para propagar etiquetas de manera claramente ventajosa. Además, el predominio de la clase $\leq 50K$ afectó la capacidad del modelo para reconocer adecuadamente la clase minoritaria.

Finalmente, los resultados confirman que no es recomendable evaluar estos modelos únicamente con accuracy. En datasets desbalanceados, métricas como recall, precision y F1-score permiten entender mejor el comportamiento real de los algoritmos. Para trabajos futuros, sería conveniente repetir el experimento con distintos porcentajes de datos etiquetados, como 5%, 10% y 20%, y evaluar técnicas adicionales de balanceo para analizar si los modelos semi-supervisados mejoran su capacidad de generalización.

Anexos

- Repositorio del proyecto: https://github.com/Bamo0507/Laboratorio10_SSL

Referencias

- Kaggle. *Adult Income Prediction Dataset*. Disponible en: <https://www.kaggle.com/datasets/mosapabdelghany/adult-income-prediction-dataset>
- UCI Machine Learning Repository. *Adult*. Disponible en: <https://archive.ics.uci.edu/dataset/2/adult>